

Predicting Software Defects from Static Code Metrics: A Case Study on KC1/JM1 Dataset

Md. Ibtihaazaman
Dept. of CSE
American International University
Bangladesh
22-49153-3@student.aiub.edu

Md. Ahasan Habib Alif
Dept. of CSE
American International University
Bangladesh
22-47516-2@student.aiub.edu

Abhijit Bhowmik
Dept. of CSE
American International University
Bangladesh
abhijit@aiub.edu

Abstract—Software bugs are a significant issue in the IT sector as they may lead to system crashes, increased maintenance expenses, and negative user experiences. Early detection of areas that have a tendency to be defective during the initial stages of the development process can help in reducing the time and cost that would otherwise be spent in many tests and correcting of defects. This paper examines the ability to predict software defects based on the use of measures of static code such as Lines of Code (LOC), cyclomatic complexity and measures of coupling. In the study, machine learning models are tested to determine defect-prone modules using the well-known KC1 and JM1 datasets. The findings reveal that the Random Forest performed better as it had 86.6% and 84.6% accuracy on the KC1 and JM1 datasets, respectively. In addition to accuracy, Random Forest also outperformed Logistic Regression in terms of ROC-AUC and PR-AUC. Key code measures of the study are key measures of code such as the LOC and cyclomatic complexity which are important measures of code to predict defects and provide useful insights to improve software quality assurance. This is a straightforward and simple to duplicate method which gives the developers a viable means of handling resources more effectively and minimize bugs in software.

Index Terms—Software Defects, Static Code Metrics, Defect Prediction, Machine Learning Models, Software Quality.

I. INTRODUCTION

Software defect prediction is a significant field in software engineering since it assists in determining which sections of the system have higher chances of having a bug prior to the release of the software. A software malfunction may lead to crashing of systems, wrong output, security and additional maintenance efforts. Software projects generally have time and resource constraints, which makes it impractical to test all the modules with the same load. It is against this reason that defect prediction can be used to help maintain quality by indicating risky modules such that the developers can focus more on code review, testing and debugging on these modules.

This research presentation is based on the prediction of defects by means of the metrics of the code that is not dynamic. Statistical measurements are the measurements which are in numeric form taken out of the source code without running the program. These are size related metrics like Lines of Code (LOC), measures of complexity like the cyclomatic

complexity and design-related metrics like coupling measures and inheritance measures. These metrics are practical in the sense that the metrics can be obtained early in development and are usually indicative of the intricacy or interdependence of a module. Generally, the greater the complexity and the greater the coupling, the more likelihood of errors and those modules will be defect prone.

The datasets used in the current study are KC1 and JM1 that are famous NASA PROMISE popular benchmark datasets. In each dataset, there are the software modules with the description of the software metrics in terms of the static code and the defect label that shows whether the software module is a defect or not. These datasets allow the study to be compared to the existing research and provide an opportunity to evaluate the model performance on the standard benchmarks.

In order to make the work easy to understand and repeat, this research paper suggests a simple workflow on two widely used machine learning models: the baseline model which is the Logistic Regression and the stronger comparative model which is the Random Forest. The results of the models will be compared based on the standard metrics like accuracy, precision, recall, F1-score, ROC-AUC and PR-AUC. Moreover, the analysis of the importance of features will be employed to gain an insight into the contribution of the most significant values (statistical metrics) to defect prediction. In general, the research will help offer an effective defect forecasting method that can be conveniently interpreted, applied and compared to the existing literature.

II. RELATED WORK

It is a key topic in software engineering since software defect prediction assists teams in locating flawed areas of a system at an early stage and enhance the quality of a software. The utilization of the metric of the static code is the focus of many studies since the metrics can be determined by gathering the source code and do not require the execution of the program. Typical examples are size measures (as e.g. Lines of Code), complexity measures (as e.g. cyclomatic complexity) and design measures (as e.g. coupling and inheritance measures). Such measures tend to indicate the difficulty of a given module to comprehend, maintain and test and studies

have indicated that increased complexity and poor design organization are commonly correlated with increased threat of defects [1], [2].

Much of the work on defect prediction applies to benchmark datasets, particularly the NASA PROMISE datasets including KC1 and JM1. These datasets are popular since they include defect labels and can be compared fairly between methods, as it gives them static metrics. Menzies et al. demonstrated that predictors of defects can be trained on the basis of the static code attributes and utilized to indicate risky modules to effort-driven testing [3]. Subsequently, Lessmann et al. compared a wide range of classifiers and discovered several machine learning models have the potential to do well, although performance is highly dependent on the data and evaluation configuration [4].

Critical restrictions have also been identified by researchers. One of them is the quality of data, as some NASA defect datasets might contain noise or inconsistent labelling, which can influence the quality of outcomes [5]. Class imbalance is another issue that occurs where defective modules have fewer modules than non-defective modules. This renders accuracy singularly deceptive and hence research studies suggest precision, recall, F1-score and the AUC-based metrics as a way of providing an accurate analysis [6]. Models Logistic Regression is commonly applied as a simple baseline, whereas Random Forest is frequently used due to its capability to account non-linear trends and feature interactions. According to the benchmark results, ensemble models such as the Random Forests can frequently have a high performance, but the results may differ in accordance with data nature [5]. Ensemble strategies are also applied to enhance performance with imbalance in some of the recent work [7]. The issue of generalization also exists: Turhan et al. discovered that cross-project training can be poorly transferred because of the differences between development practices and data distribution [8]. Recent studies examine other indicators than the metrics of codes, like the tracking of the developers task behavior, though might be more difficult to implement in practice [9].

In general, there is evidence that supports defect prediction using static-metrics, however, issues of dataset quality, imbalance and reproducibility exist. The proposed study considers these problems by applying a clear baseline workflow to the Logistic Regression and Random Forest and assessing it with several standard measures.

III. METHODOLOGY

The proposed study follows a quantitative experimental design using supervised machine learning to predict whether a software module is defective or non-defective. The study uses the NASA PROMISE KC1 and JM1 datasets [13], where each software module is represented by a set of static code metrics (e.g., size, complexity, coupling, inheritance measures) and a defect label. The main goal is to build a clear and reproducible workflow and compare two widely used models: Logistic Regression (baseline) and Random Forest (comparative model).

A. Participants and Sampling

The unit of analysis in this research is software modules/classes in the KC1 and JM1 datasets. The dataset consists of rows that describe software modules, and the target label is the presence or absence of a defect in the software module. No participant recruiting is necessary since the current research involves the utilization of publicly available records of the dataset and does not require the involvement of human subjects. The modules are taken separately in terms of training and evaluation.

B. Data Preprocessing

The datasets are cleaned and ready to be used before being trained to produce the models. First, the data is verified for missing, invalid, and duplicate records. In case of missing values, they are treated in simple ways like dropping rows with excessive missing values or performing simple imputation where necessary. Then, the defect label is brought into binary representation (defect = 1, non-defect = 0).

As most metrics of the code are usually not in the same range, feature scaling (standardization) is applied in the Logistic Regression model to ensure that large-scale features do not overshadow their smaller-scale counterparts. Random Forest does not require scaling, but the same cleaned set of features is used in both models to ensure a fair comparison. In case of extreme outliers, they are checked and handled cautiously to prevent bias during model training.

C. Train-Test Split

In order to test the models correctly, the data was separated into the training set (70 percent) and the testing set (30 percent). A stratified division was employed to ensure that the number of defective and non-defective modules is closer in both sets. This minimizes the level of bias and it aids in ensuring that the results of the test are a reflection of actual performance and not some random sampling effects.

D. Handling Class Imbalance

Defect datasets commonly suffer from class imbalance, where defective modules are fewer than non-defective modules. If not handled, models may become biased toward predicting the majority class, reducing defect detection ability. To address this issue, class weighting (balanced mode) is used so that misclassifying defective modules carries a higher penalty during training. Evaluation focuses on metrics such as precision, recall, F1-score, ROC-AUC and PR-AUC because accuracy alone is not reliable under imbalance.

E. Model Development

- Logistic Regression (Baseline Model): Logistic Regression is selected as the baseline due to its simplicity, commonness, and interpretability. It provides probabilities and helps elucidate the contribution of metrics through model coefficients.
- Random Forest (Comparative Model): Random Forest is chosen in that it can identify non-linear relationships and

interactions among metrics. It generally works well on structured data as well as feature importance, which is useful to determine the influential metrics.

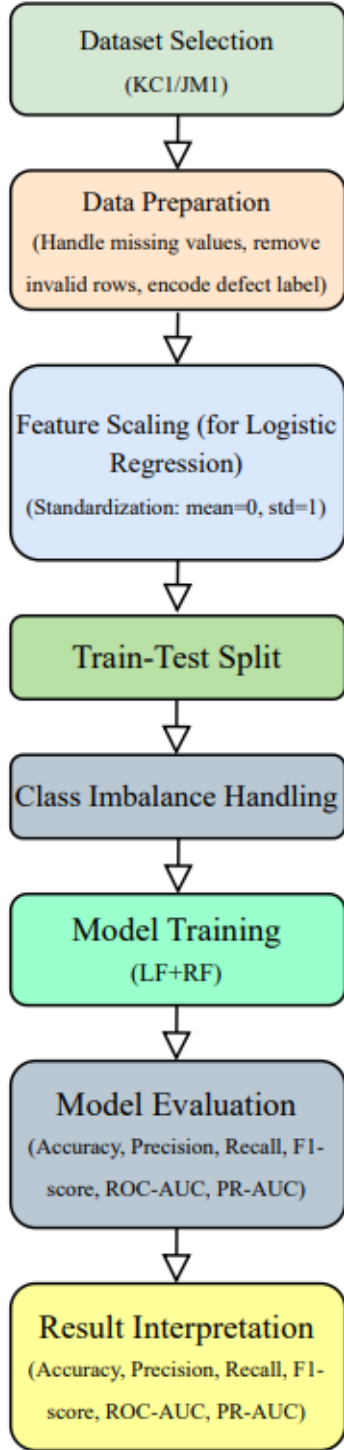


Fig. 1. Workflow of the proposed defect prediction study procedure (KC1/JM1)

This workflow diagram presents the complete research pipeline.

IV. RESULTS AND DISCUSSION

A. Evaluation Method

The performance measures standard performance measures was used to evaluate the models on the test set:

- Accuracy (Overall Correctness)
- Precision (number of predicted defective modules which are actually)
- Recall (how many real faulty modules are identified)
- F1-score (trade-off between accuracy and recall)
- ROC-AUC (ranking ability over thresholds)
- PR-AUC (helpful when there is an imbalance in the classes)

In illustrating performance in a model comparison, this is the form of a bar chart and ROC chart (similar to the way it was presented in earlier semester reports).

B. Model Result

TABLE I
MODEL RESULTS (KC1 AND JM1)

Dataset	Model	Accuracy	Precision	Recall	F1-score	ROC-AUC	PR-AUC
KC1	Severity	Logistic Regression	0.739	0.333	0.684	0.448	0.807
KC1	Random Forest	0.866	0.633	0.316	0.422	0.825	0.502
JM1	Logistic Regression	0.722	0.302	0.569	0.395	0.723	0.37
JM1	Random Forest	0.846	0.543	0.192	0.283	0.78	0.421

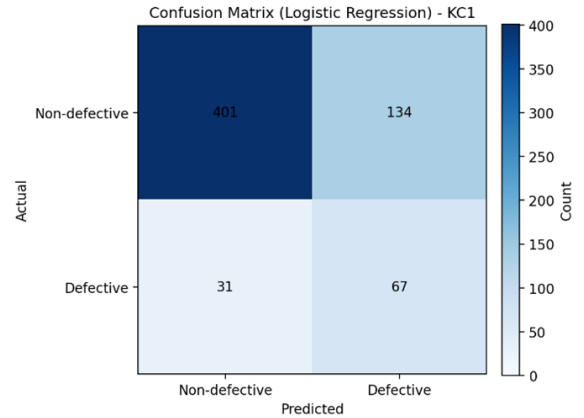


Fig. 2. Confusion Matrix - KC1 (Logistic Regression)

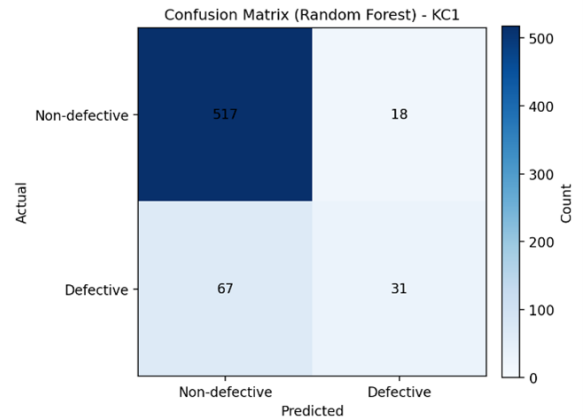


Fig. 3. Confusion Matrix - KC1 (Random Forest)

KC1 (Logistic Regression): This confusion matrix shows how Logistic Regression classifies KC1 modules into defective and non-defective classes. It highlights correct predictions (TP and TN) and misclassifications (FP and FN) on the test set. KC1 (Random Forest): This figure presents the Random Forest classification results on KC1 using a confusion matrix. It indicates how many defective modules were correctly detected and how many errors occurred, providing an error-based view of performance.

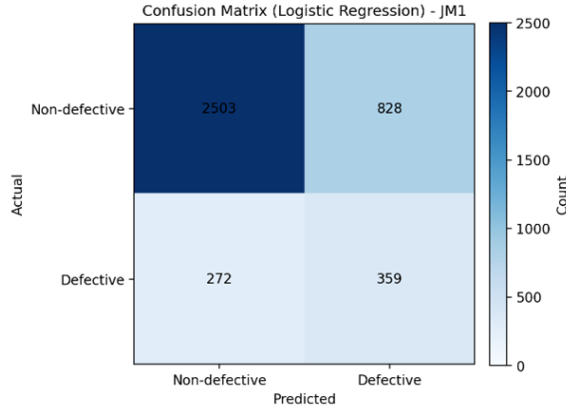


Fig. 4. Confusion Matrix - JM1 (Logistic Regression)

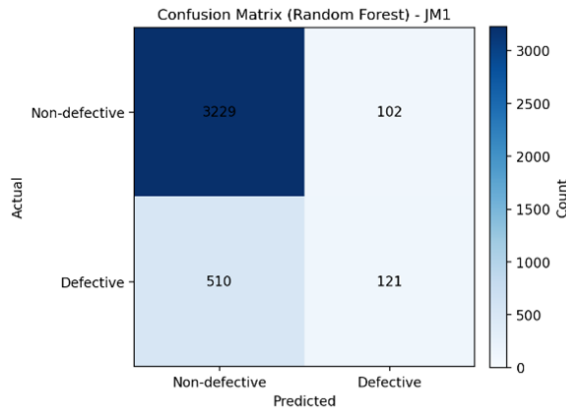


Fig. 5. Confusion Matrix - JM1 (Random Forest)

JM1 (Logistic Regression): This confusion matrix summarizes Logistic Regression predictions on the JM1 dataset. It shows correct classifications and mistakes, which helps understand whether the model misses defects (FN) or raises false alarms (FP). JM1 (Random Forest): This figure shows the confusion matrix for Random Forest on JM1 and explains the distribution of correct and incorrect predictions. It helps evaluate defect detection ability beyond overall accuracy, especially under class imbalance.

This figure shows the precision-recall trade-off across different thresholds and is especially informative when the datasets are class-imbalanced.

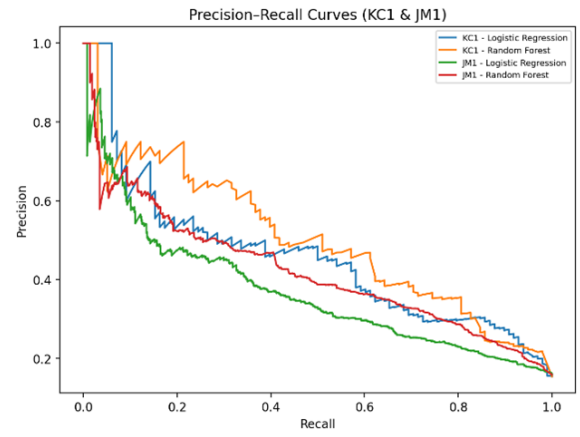


Fig. 6. Precision-Recall Curves (KC1 and JM1)

This figure shows the precision-recall trade-off across different thresholds and is especially informative when the datasets are class-imbalanced.

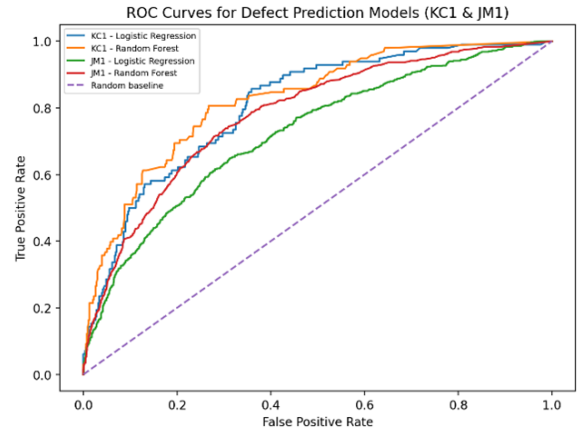


Fig. 7. ROC Curves for defect prediction models on KC1 and JM1 (Test Sets)

ROC Curves for defect prediction models on KC1 and JM1 (Test Sets). This figure illustrates the true positive rate vs false positive rate across thresholds; higher curves and larger AUC indicate better class separation.

C. Feature Importance

Finally, the study was analyze which static metrics contribute most to defect prediction. For Random Forest, feature importance was used to rank the top predictors. For Logistic Regression, model coefficients (after scaling) was interpreted to understand which metrics increase or decrease defect risk. This step is important because the study aims not only to predict defects, but also to explain which code properties are most related to defect-proneness.

KC1 dataset: This figure ranks the most influential static code metrics used by Random Forest to predict defect-prone modules in KC1. Higher importance indicates a stronger contribution to the model's decisions. JM1 dataset: This figure

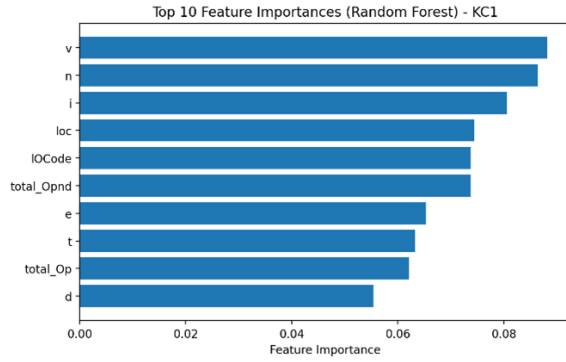


Fig. 8. Top 10 feature importances (Random Forest) for KC1

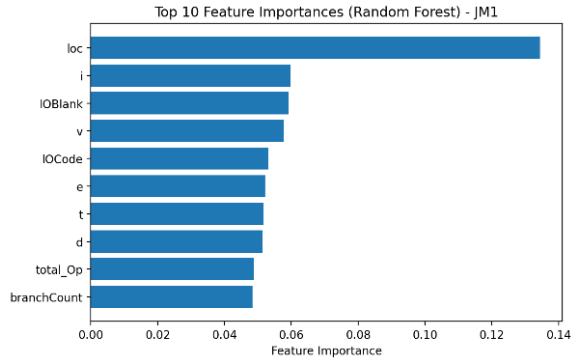


Fig. 9. Top 10 feature importances (Random Forest) for JM1

shows which static metrics contribute most to defect prediction in JM1 according to Random Forest. It supports interpretation by indicating the metrics that drive predictions.

V. CONCLUSION AND FUTURE DIRECTIONS

This work demonstrates that the NASA PROMISE KC1 and JM1 datasets of static code metrics can be effectively used to support software defect prediction with the help of supervised machine learning. Both the Logistic Regression and random forest demonstrated reasonable performance in terms of accuracy, F1-score, ROC-AUC and PR-AUC even when the class imbalance is very high. Logistic Regression was more likely to pick more defected modules, whereas the Random Forest was more likely to give a higher precision and overall accuracy to detect more defected modules and avoiding false alarms. As these measures are not complicated to gather and analyze, the suggested workflow may be incorporated into actual projects to emphasize the risky modules and intensive testing shall be performed before the risky module is initiated.

Future research would investigate the more advanced models, e.g. gradient boosting, XGBoost or deep forest models, to determine whether they can outperform the current baselines on KC1 and JM1. Other methods of dealing with class imbalance, such as various resampling approaches or hybrid cost-sensitive approaches, might be considered to help increase the recall and F1-score of defective modules. Cross-project

experiments with models being trained on one set of data and tested on a different set would help evaluate the extent to which defect predictors are applicable across different software systems. It would also be useful to consider combining the measures of the static code with the process and developer-related measures, such as change history or commit activity, to have a richer picture of defect risk..

REFERENCES

- [1] S. R. Chidamber and C. F. Kemerer, "A Metrics Suite for Object-Oriented Design," *IEEE Trans. Softw. Eng.*, 1994. doi: 10.1109/32.295895.
- [2] R. Subramanyam and M. S. Krishnan, "Empirical Analysis of CK Metrics for Object-Oriented Design Complexity: Implications for Software Defects," *IEEE Trans. Softw. Eng.*, 2003. doi: 10.1109/TSE.2003.1191795.
- [3] T. Menzies, J. Greenwald, and A. Frank, "Data Mining Static Code Attributes to Learn Defect Predictors," *IEEE Trans. Softw. Eng.*, 2007. doi: 10.1109/TSE.2007.10.
- [4] S. Lessmann, B. Baesens, C. Mues, and S. Pietsch, "Benchmarking Classification Models for Software Defect Prediction: A Proposed Framework and Novel Findings," *IEEE Trans. Softw. Eng.*, 2008. doi: 10.1109/TSE.2008.35.
- [5] T. Hall, S. Beecham, D. Bowes, D. Gray, and S. Counsell, "A Systematic Literature Review on Fault Prediction Performance in Software Engineering," *IEEE Trans. Softw. Eng.*, 2012. doi: 10.1109/TSE.2011.103.
- [6] M. Shepperd, Q. Song, Z. Sun, and C. Mair, "Data Quality: Some Comments on the NASA Software Defect Datasets," *IEEE Trans. Softw. Eng.*, 2013. doi: 10.1109/TSE.2013.11.
- [7] B. Rupadevi and R. Chandana, "Predicting Software Defects with Smart Ensemble Learning Techniques," *International Research Journal of Innovations in Engineering and Technology (IRJIET)*, vol. 9, Special Issue ICCIS-2025, pp. 117-123, May 2025. doi: 10.47001/IRJIET/2025.ICCIS-202519.
- [8] B. Turhan, T. Menzies, A. B. Bener, and J. D. Stefano, "On the Relative Value of Cross-Company and Within-Company Data for Defect Prediction," *Empirical Softw. Eng.*, 2009. doi: 10.1007/s10664-008-9103-7.
- [9] G. Laudato, S. Scalabrino, N. Novielli, F. Lanubile, and R. Oliveto, "Predicting Bugs by Monitoring Developers During Task Execution," in *Proc. 45th Int. Conf. Software Engineering (ICSE)*, 2023. doi: 10.1109/ICSE48619.2023.00100.
- [10] S. Omri, P. Montag, and C. Sinz, "Static Analysis and Code Complexity Metrics as Early Indicators of Software Defects," *J. Softw. Eng. Appl.*, 2018. doi: 10.4236/jsea.2018.114010.
- [11] J. Nam, W. Fu, S. Kim, T. Menzies, and L. Tan, "Heterogeneous Defect Prediction," *IEEE Trans. Softw. Eng.*, 2018. doi: 10.1109/TSE.2017.2720603.
- [12] T. Zhou, X. Sun, X. Xia, B. Li, and X. Chen, "Improving Defect Prediction with Deep Forest," *Information and Software Technology*, 2019. doi: 10.1016/j.infsof.2019.07.003.
- [13] ApoorvaKrisna, "NASA-promise-dataset-repository," GitHub repository, 2020. [Online]. Available: <https://github.com/ApoorvaKrisna/NASA-promise-dataset-repository>.